

# From Scheduled Execution to Itinerant Dynamics: Three Semantics for Content-Addressable Recurrent Operator Machines

Working note

July 2026

## Abstract

CAROM (Content-Addressable Recurrent Operator Machine) is an architecture that separates control from data: opaque command tokens route computation over a small library of reusable learned operators, which read, transform, gate, and write a register-like slot workspace over a fixed sequence of recurrent steps. This note develops two successive reformulations of its execution model. First, a *fixed-point* (deep-equilibrium) semantics replaces the externally imposed schedule with joint constraint satisfaction: the answer is the workspace configuration self-consistent under all commands at once. This buys  $O(1)$  training memory, adaptive computation, and bidirectional inference, at the cost of the directionality that makes composition work. Second, an *itinerant* semantics recovers directionality without reintroducing a clock: commands compete for control of the workspace through a learned generalized Lotka–Volterra dynamic, and sequential order emerges as a stable heteroclinic channel through metastable states. The scheduled and fixed-point models are exact special cases of the itinerant one, obtained respectively by clamping and by symmetrizing a single learned inhibition matrix. We give the coupled equations, the stability conditions under which the construction is trainable, and a set of discriminating experiments.

## 1 CAROM: a differentiable register machine

### 1.1 Intuition

CAROM is best understood as a CPU metaphor made differentiable. A CPU separates three things that a Transformer entangles: *instructions* (which computation to perform), *registers* (named locations whose contents evolve), and *circuits* (a small reusable set of functional units that many instructions share). CAROM reproduces this factorization. Command tokens are instructions: they never touch data directly, they only select circuitry. Workspace slots are registers: each has a fixed address, an immutable record of its initial contents, and a mutable current value. Operators are circuits: a small library of learned read–transform–gate–write units, shared across all commands, applied to all slots in parallel.

The bet is that this factorization is the right inductive bias for *compositional* computation. If commands can only act through a shared operator library, command semantics are forced to be combinations of reusable primitives; if slots separate address from value, variable binding is structural rather than learned. Empirically the architecture performs roughly on par with a Transformer on generic sequence tasks and shows an advantage when the task is to learn composable functions over sequences — i.e., the bias pays exactly where it was aimed.

## 1.2 Formal summary

The input is a sequence containing  $C$  command tokens  $c_1, \dots, c_C$  and  $n$  data tokens  $x_1, \dots, x_n$ . Only data tokens become workspace slots. Slot  $i$  carries three vectors in  $\mathbb{R}^d$ :

$$\begin{aligned} e_i &= \text{Norm}(E_{\text{tok}}(x_i)) && \text{(immutable evidence),} \\ p_i &= E_{\text{pos}}(i) && \text{(fixed address),} \\ w_i^t &\in \mathbb{R}^d, \quad w_i^0 = e_i && \text{(mutable content after } t \text{ updates).} \end{aligned}$$

**Command-only routing.** A learned table  $A \in \mathbb{R}^{C \times K}$  assigns each command a softmax mixture over  $K$  operators,

$$\rho_k(c) = \frac{\exp(A_{c,k}/\tau_r)}{\sum_{j=1}^K \exp(A_{c,j}/\tau_r)}, \quad (1)$$

depending only on command identity. Distinct commands may share operators; with  $K$  small relative to the diversity of commands, the table is pressured to become a reusable instruction set.

**One operator.** Operator  $k$  is a full micro-circuit acting on all slots in parallel. Reads are content-addressed attention over source slots  $j$ , with queries/keys/values built from  $[w; e; p]$  so that current content, original evidence, and address all participate in binding:

$$r_i^{t,k} = \sum_{j=1}^n a_{ij}^{t,k} v_j^{t,k}, \quad a_{ij}^{t,k} = \text{softmax}_j \left( \frac{(q_i^{t,k})^\top k_j^{t,k}}{\sqrt{d}} \right). \quad (2)$$

An operator-specific, command-independent transform  $\phi_k$  produces  $z_i^{t,k} = \phi_k(r_i^{t,k}, w_i^t, e_i, p_i)$ ; a sigmoid gate  $g_i^{t,k} = \sigma(W_G^k z_i^{t,k} + b) \in (0, 1)^d$  and a SwiGLU $_k$  head yield a gated update proposal  $u_i^{t,k}$ .

**Scheduled mixing and recurrence.** A fixed monotonic one-hot schedule  $\gamma^t \in \{e_1, \dots, e_C\}$  activates one command per recurrent step; the active command determines the operator mixture

$$\beta_k^t = \sum_{m=1}^C \gamma_m^t \rho_k(c_m), \quad w_i^{t+1} = w_i^t + \sum_{k=1}^K \beta_k^t u_i^{t,k}, \quad (3)$$

for  $t = 0, \dots, T-1$ . After  $T$  steps a per-slot readout  $\hat{y}_i = \arg \max_y [W_{\text{out}} \text{Norm}(\phi_o(w_i^T, e_i, p_i))]_y$  produces the output sequence; supervision applies only there.

## 1.3 What is right and what binds

*Strengths.* Structural variable binding (address/evidence/content decomposition); a learned reusable operator library; direct interpretability (the routing table  $A$  is a readable program dictionary,  $\beta^t$  is an execution trace); stable residual gated recurrence; and the empirically observed compositional advantage.

*The binding constraint* is Eq. (3) itself. Three costs follow from the forced schedule. (i) *Sequentiality:*  $T$  strictly sequential steps, with  $O(T)$  BPTT memory and  $O(T)$  wall-clock depth,

against a Transformer’s parallel training. (ii) *Static control flow*: the mixture  $\beta^t$  is fully determined before any data is seen; there are no conditionals, no loops, no data-dependent dispatch, and the same command always routes identically regardless of context. (iii) *Fixed budget*:  $T$  is a hyperparameter with no halting mechanism. The remainder of this note is about relaxing Eq. (3) while keeping everything else.

## 2 The fixed-point reformulation

### 2.1 Intuition: from procedure to constraint

Scheduled CAROM defines the answer *procedurally*:  $w^T$  is whatever executing the commands in order produces. The fixed-point reformulation defines it *declaratively*: the answer is the workspace configuration that is self-consistent under all commands simultaneously. Each command stops being an instruction executed at a moment and becomes a standing constraint — “slot contents should relate thus” — and inference is relaxation to the configuration on which all constraints agree. For a content-addressable workspace this is arguably the more natural semantics: the workspace settles into the interpretation jointly licensed by the commands. It is the denotational semantics of the same program whose operational semantics is Eq. (3).

### 2.2 Formulation

Define the simultaneous update field

$$F(w) = w + \sum_{m=1}^C \sum_{k=1}^K \rho_k(c_m) u^{(k)}(w, e, p, s_m), \quad (4)$$

where  $s_m$  is a per-command *step embedding* carrying ordinal information into the operators’ queries and keys (order survives as a feature, not as time). The model’s output is an equilibrium

$$w^* = F(w^*), \quad (5)$$

found by damped iteration  $w \leftarrow (1 - \alpha)w + \alpha F(w)$ , Anderson acceleration, or a quasi-Newton solver, initialized at  $w^0 = e$ . Readout applies to  $w^*$ .

**Training.** By the implicit function theorem, gradients require no backpropagation through the solver trajectory:

$$\frac{\partial \ell}{\partial \theta} = \frac{\partial \ell}{\partial w^*} \left( I - \frac{\partial F}{\partial w} \Big|_{w^*} \right)^{-1} \frac{\partial F}{\partial \theta} \Big|_{w^*}, \quad (6)$$

with the inverse applied by a cheap fixed-point iteration on vector–Jacobian products (the deep-equilibrium recipe). Consequences:  $O(1)$  activation memory independent of solver depth; no gradient path through “time” because there is none; and solver-independence — train with one solver, deploy with another, since the gradient is a property of the equilibrium, not the path.

**Well-posedness.** Existence, useful uniqueness, and reachability of  $w^*$  require  $F$  to be (locally) contractive. CAROM’s components help — sigmoid gates bound update magnitudes, normalization controls scale — but in practice one adds spectral normalization or a Lipschitz penalty on the operators, damping  $\alpha < 1$ , and a Jacobian regularizer penalizing an estimate

of  $\|\partial F/\partial w\|$  at the equilibrium. Failure modes if omitted: multiple equilibria (answers depend on initialization), limit cycles, or stiff equilibria where the backward solve in Eq. (6) dominates cost.

## 2.3 Why this improves scheduled CAROM

- (a) *Parallelism and memory.* Each solver sweep updates all slots under all commands simultaneously; sequential depth becomes “iterations to convergence,” a property of problem hardness rather than program length, and training memory is  $O(1)$  by Eq. (6).
- (b) *Adaptive computation for free.* Iterate to a residual tolerance: easy inputs converge fast, hard inputs take more sweeps, and test-time compute scales cleanly — more iterations buy a better answer to the same well-defined problem.
- (c) *Bidirectional inference.* A late constraint under pressure propagates residual back and revises earlier slots. Scheduled execution structurally forbids this; parsing-like, abductive, and error-correcting tasks require it.
- (d) *A confidence channel.* Per-slot residual magnitude at budget reports which parts of the constraint system failed to settle.

Note also that Jacobi iteration of  $F$  from  $w^0 = e$  *simulates* forward execution: in sweep one, only constraints grounded in raw evidence act; in sweep two, constraints depending on those results activate; and so on, so information traverses the dependency DAG in critical-path-many sweeps.

## 2.4 What it loses

Composition is directional: in  $f(g(x))$ ,  $g$ ’s result must inform  $f$  and not conversely. The schedule provided this by temporal order; the fixed point must encode it through the learned asymmetries induced by the step embeddings  $s_m$ , and whether learned asymmetry suffices is an open empirical question. More fundamentally, a single global equilibrium has no internal phases: it cannot represent “first establish  $X$ , then, in the context of  $X$ , establish  $Y$ ” except by folding both into one simultaneous constraint set. This motivates the next section.

# 3 Itinerant dynamics: order without a clock

## 3.1 Intuition

The scheduled model imposes order from outside; the fixed-point model deletes it. The itinerant model lets order *emerge*. The device is to promote the schedule from an input to a dynamical variable: introduce a low- dimensional control state in which commands compete for control of the workspace. The winning command’s operators push the workspace toward local consistency (a phase of constraint satisfaction); the changed workspace alters the competitive fitness of the other commands; a successor takes over. Computation becomes a trajectory through a chain of metastable way-stations — linger, work, hand off — and the visitation order is carved into a learned inhibition structure rather than clocked in. Because a command’s fitness reads the workspace, the itinerary is data-dependent: conditionals, loops, and halting arise as dynamical

phenomena. Within each phase one has equilibrium semantics (bidirectional constraint propagation); across phases, directional composition. The two semantics of Sections 1–2 coexist at different timescales of a single flow, and each is recovered exactly as a special case.

The relevant dynamical object is the *stable heteroclinic channel* (Afraimovich–Rabinovich): a chain of saddle equilibria connected by heteroclinic orbits which is itself an attracting set. Unlike a strange attractor, an SHC has no positive Lyapunov exponents — trajectories contract transversally into the channel — so the construction stays on trainable ground while occupying far wider computational territory than a fixed point.

### 3.2 Formulation

Augment the workspace  $w \in \mathbb{R}^{n \times d}$  with a control state  $a \in \mathbb{R}_{\geq 0}^M$  over  $M$  modes: one mode per command instance plus a distinguished halt mode. The coupled stochastic flow is

$$\frac{da_m}{dt} = a_m \left( \sigma_m(w) - \sum_{j=1}^M \rho_{mj} a_j \right) + \eta_a \xi_m(t), \quad (7)$$

$$\frac{dw_i}{dt} = -\lambda w_i + \sum_{m=1}^M a_m(t) \sum_{k=1}^K \rho_k(c_m) u_i^{(k)}(w, e, p) + \eta_w \zeta_i(t). \quad (8)$$

Eq. (8) is the CAROM update field of Section 1 — same operator library, same routing table — with mixture weights  $\beta_k(t) = \sum_m a_m(t) \rho_k(c_m)$  now supplied by the control dynamics. Eq. (7) is generalized Lotka–Volterra competition: mode  $m$  grows at a rate  $\sigma_m(w)$  — a learned readout of the current workspace, the locus of data-dependence — and is suppressed by other modes through the learned inhibition matrix  $\rho \in \mathbb{R}^{M \times M}$ . Small noise terms  $\xi, \zeta$  play a functional role given below. Optionally each mode carries a slow fatigue variable

$$\tau_f \frac{df_m}{dt} = a_m - f_m, \quad \sigma_m(w) \mapsto \sigma_m(w) - \kappa f_m, \quad (9)$$

which penalizes prolonged occupancy and guarantees motion.

#### Containment of the earlier variants.

**Proposition 1** (Special cases). *(i) Clamping  $a(t)$  to the externally forced one-hot sequence  $\gamma^t$  reduces Eqs. (7)–(8) to scheduled CAROM, Eq. (3), in discretized time. (ii) If  $\rho$  is symmetric (and  $\sigma$  workspace-independent on the relevant region), the competition Eq. (7) admits a Lyapunov function and  $a(t)$  converges to a stable mixture  $a^*$ ; the workspace flow then relaxes to the equilibrium of the static field  $F(\cdot; \beta(a^*))$ , recovering the fixed-point model of Section 2.*

Thus the symmetry of one learned  $M \times M$  matrix is the dial between declarative and procedural semantics. One may regularize  $\|\rho - \rho^\top\|$  toward zero for constraint-like tasks, toward a target chain for program-like tasks, or leave it free and observe which semantics the task induces.

**The itinerancy mechanism.** In GLV competition the pure states “only mode  $m$  active” are equilibria; the entries of  $\rho$  decide whether each is a sink or a saddle. The Afraimovich–Rabinovich conditions state, qualitatively: let each mode inhibit its designated successor weakly enough that the successor can invade, let successors inhibit predecessors strongly enough to

extinguish them, and suppress all other competitors hard. Then the pure states become saddles chained by heteroclinic connections, and the chain is attracting — a stable heteroclinic channel — provided each saddle is dissipative:

$$\nu_m = \frac{|\lambda_m^{\text{stable}}|}{\lambda_m^{\text{unstable}}} > 1 \quad \text{at every station } m. \quad (10)$$

Trajectories are funneled into the channel, dwell near each saddle, and pass onward in the prescribed order. The inhibition matrix is therefore the *program-order prior*: a strict chain encodes a fixed sequence; a DAG encodes partial order with data-dependent resolution ( $\sigma_m(w)$  breaks ties); a cycle encodes iteration, exited when the workspace evolves to where some  $\sigma_{\text{exit}}$  dominates. Noise sets the clock: near a saddle with unstable rate  $\lambda_u$ , mean exit time scales as  $\ln(1/\eta_a)/\lambda_u$ , so dwell time per phase is finite, tunable, and roughly uniform along the channel. Fatigue, Eq. (9), is the complementary mechanism: asymmetric inhibition proposes the itinerary, fatigue guarantees no station becomes a trap.

**Halting and readout.** The halt mode has learned growth rate  $\sigma_{\text{halt}}(w)$  — a “done” detector on the workspace — and is made absorbing ( $\rho_{\text{halt},j}$  large for all  $j$ , nothing inhibits it strongly). Readout fires on  $w$  when  $a_{\text{halt}}$  crosses threshold: variable-length computation with learned halting and no auxiliary halting machinery. A second output channel comes free: thresholding  $a(t)$  yields the itinerary — the sequence of saddles visited with dwell times — an emergent, data-dependent program trace, more informative than the scheduled model’s trace precisely because it was chosen rather than copied from the input.

### 3.3 Timescale separation and training

Take the workspace fast and the control slow ( $\lambda$  large relative to the scale of Eq. (7)). Then within each metastable phase,  $w$  approximately equilibrates under the currently dominant mixture: each phase is a small deep-equilibrium solve, differentiable by Eq. (6) at  $O(1)$  memory. The control trajectory  $a(t)$  is low-dimensional ( $M \ll nd$ ) and is backpropagated directly — pathwise gradients through the reparameterized noise — at negligible cost. Training is thus a *chain of implicit solves stitched by a cheap explicit trajectory*: no BPTT through the workspace history. Because the SHC has no positive Lyapunov exponents, the exploding and ill-defined gradients of chaotic regimes do not arise; the sensitive quantities are saddle passage times (logarithmic in noise, hence soft) and the discrete itinerary near decision boundaries of  $\sigma_m(w)$ , handled by temperature on the  $\sigma$  readouts annealed toward sharpness.

### 3.4 Why itinerancy improves on the fixed point

- (a) *Directionality restored, structurally.* Phase order is a property of the channel, not a learned feature hint: “first  $X$ , then  $Y$  in the context of  $X$ ” is representable as two saddles, which no single global equilibrium expresses.
- (b) *Data-dependent control flow.* Since  $\sigma_m$  reads  $w$ , a command invades only when its pre-condition pattern is present — content-triggered dispatch, which command-identity-only routing forbids — and cycles in  $\rho$  with workspace-dependent exit give genuine loops.
- (c) *Both semantics at once.* Within a phase: equilibrium relaxation, bidirectional constraint propagation, error correction. Across phases: directional composition. The fixed point offers only the former; the schedule only the latter.

- (d) *Learned halting and emergent traces*, as above; neither exists in the fixed-point model except through solver tolerance.
- (e) *Strict containment*. By Proposition 1, nothing is lost: the model can collapse to either predecessor where a task rewards it, and which collapse a task induces is itself an experimental observable.

### 3.5 Failure modes and regularizers

*Channel collapse* (a saddle becomes a sink; computation stalls): guaranteed against by fatigue, or monitored via saddle eigenvalues. *Channel evaporation* ( $\nu_m < 1$  somewhere; trajectories leak): regularize Eq. (10) with a margin along realized itineraries. *Phase smearing* ( $a$  stays diffuse; the soft-mixture blur returns): entropy penalty on  $a$  encouraging near-vertex dwell. *Itinerary instability* (noise resamples yield different routes on the same input): penalize itinerary variance across resamples on deterministic tasks — while noting that on genuinely ambiguous inputs route variance is the model sampling interpretations, a feature connected to stochastic- attractor readouts rather than a defect.

## 4 Experiments

Throughout, evaluate where the inductive bias lives: compositional splits, program execution, length and depth generalization — not perplexity.

**Experiment 1** (Localizing the value of recurrence). Unroll scheduled CAROM into  $L$  feed-forward layers sharing the operator library (schedule becomes layer assignment) and compare on the compositional tasks where CAROM beat the Transformer. If the advantage survives unrolling, the command/operator factorization and register workspace carry it and the cheap parallel version ships; if not, recurrence is load-bearing and the reformulations of Sections 2–3 are the right investment.

**Experiment 2** (Forward composition vs. backward revision). Construct function-composition tasks in two regimes: (i) pure forward chains, and (ii) chains containing an embedded inverse problem, solvable only by propagating a late constraint back to revise earlier slots. Predictions: scheduled CAROM matches the fixed-point model on (i), with the fixed-point model’s sweep count tracking critical-path depth (measurable and falsifiable); the fixed-point model pulls ahead on (ii). Failure of the fixed-point model on (i) indicates that step-embedding asymmetry is insufficient and motivates causal masking of the constraint graph.

**Experiment 3** (Emergent order). Present composed functions with commands in *scrambled* order, so correct execution order must be discovered from data. Scheduled CAROM is structurally handicapped; the fixed-point model must absorb order into the equilibrium; the itinerant model can express order but must learn it. Success criterion: extracted itineraries (thresholded  $a(t)$ ) match the ground-truth dependency order on held-out compositions. This is the sharpest test of the central claim — that a network’s trajectory through state space can be a readable, data-dependent program.

**Experiment 4** (Loops and halting). Tasks with data-dependent iteration counts (e.g., iterate a map until a predicate holds; string rewriting to normal form). Measure: halt-mode timing accuracy; generalization to iteration counts beyond the training range, where scheduled CAROM

cannot even represent the larger programs; dwell-time statistics against the  $\ln(1/\eta_a)/\lambda_u$  prediction.

**Experiment 5** (Which semantics does the task induce?). Train the itinerant model with  $\rho$  unregularized on (i) constraint- satisfaction tasks and (ii) sequential-algorithmic tasks; track the asymmetry  $\|\rho - \rho^\top\|$  and the Lyapunov spectrum of the realized dynamics. Prediction: (i) drives  $\rho$  toward symmetry (equilibrium semantics rediscovered), (ii) toward chain asymmetry (procedural semantics rediscovered). Observing task-dependent selection of computational semantics within one architecture would be a more interesting result than either endpoint alone.

**Experiment 6** (Ablations on the itinerancy mechanism). (a) Fatigue only vs. asymmetric inhibition only vs. both, testing the claim that asymmetry proposes order while fatigue prevents traps. (b) Noise amplitude sweeps, testing dwell-time control and the deterministic limit. (c)  $\sigma_m$  made workspace-independent, ablating data-dependent dispatch: conditionals and loops should fail selectively.

## 5 Closing remark

The three models are one architecture under three execution semantics — operational (Section 1), denotational (Section 2), and itinerant (Section 3) — sharing a single operator library and routing table. The itinerant formulation contains the other two as limits of one learned matrix, and adds what both lack: emergent, data-dependent, interpretable control flow, realized as stable heteroclinic itinerancy rather than either an external clock or a single global equilibrium. Metastable attractor ruins as the phases of a computation, transitions as inference steps, the itinerary as the program: this is attractor dynamics as cognition, in a form that is trainable with current machinery.